Exp: 6
6/8/26

Error Detection & Correction using Hamming Code

Aim:

To write a program to implement error detection and correction using Hamming Code.

Algorithm:

1. Start the program
2. Read the number of 4 bit data blocks (n) forming the input data stream.
3. For each 4-bit data block D = d1, d2, d3, d4, perform Hamming code.

a. Arrange 7-bits with parity bits at positions 1, 2, 4, and data bits at positions 3, 5, 6, 7.

b. Compute the parity bits:

P1 = d1 XOR d2 XOR d4

P2 = d1 XOR d3 XOR d4

P4 = d2 XOR d3 XOR d4

c. From the 7-bit Hamming code word: ~~P1, P2, P3~~

P1 P2 d1 P4 d2 d3 d4.

4. Simulate transmission by introducing a single-bit error at a chosen bit position.

5. At the receiver, recompute the check bits from the received code word:

   C1 = P1 XOR d1 XOR d4

   C2 = P2 XOR d3 XOR d4

   C4 = P4 XOR d3 XOR d4

6. Compute error position = C4 C2 C1

7. If error position = 0, no error has occured. Else, the bit at "Error Position" is inverted to correct the error.

8. Extract the original 4 bits from the corrected 7-bit code word.

9. Repeat step 3 to 8 for all n blocks in the stream.

10. Stop the program.

Program:

```
def encode(d):
    h = [0]*7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print(f"Error detected at bit position: {error_pos} -> Correcting....")
        h[error_pos-1] ^= 1
    else:
        print("No error detected")
    return error_pos
```

```
def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    n = int(input("Enter number of 4-bit data blocks in the stream: "))
    for b in range(1, n+1):
        print(f"\n-- Block {b} --")
        d = list(map(int, input("Enter 4 data bits: ")))
        h = encode(d)
        print(f"Original Data bits : {to_str(d)}")
        print(f"Encoded Hamming Code (7 bits): {to_str(h)}")
        pos = int(input("Enter bit position to introduce error (1-7, 0=no error): "))
        if 1 <= pos <= 7:
            h[pos-1] ^= 1
            print(f"Transmitted Code (with error induced at pos {pos}): {to_str(h)}")
        else:
            print(f"Transmitted Code (no error induced): {to_str(h)}")
```

```
    print(f"Received Code : {to_str(h)}")
    print(f" Checking for errors ")
    detect_and_correct(h)
    print("Corrected Code : ", str(h))
    rcd = [h[2], h[4], h[5], h[6]]
    print(f"Recovered Data bits: {to_str(h)}")

if __name__ == "__main__":
    main()
```



```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits     : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (with error induced at pos 3): 0100011
Received Code          : 0100011
Checking for errors...
  Error detected at bit position: 3 -> Correcting...
Corrected Code         : 0110011
Recovered Data Bits    : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 1 1 0
Original Data Bits     : 0110
Encoded Hamming Code (7 bits): 1100110
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1100100
Received Code          : 1100100
Checking for errors...
  Error detected at bit position: 6 -> Correcting...
Corrected Code         : 1100110
Recovered Data Bits    : 0110
```

Result:

The program to implement error detection and correction using Hamming Code has been completed.